

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/272494253>

Towards a Fuzzy based Framework for Effort Estimation in Agile Software Development

Article in *International Journal of Computer Science and Information Security*, · January 2015

CITATIONS

43

READS

2,274

3 authors:



Atef T. Nour El-Din Raslan

8 PUBLICATIONS 76 CITATIONS

[SEE PROFILE](#)



Nagy Ramadan

Cairo University

115 PUBLICATIONS 773 CITATIONS

[SEE PROFILE](#)



Hesham A. Hefny

Faculty of Graduate Studies for Statistical Research (FGSSR)

320 PUBLICATIONS 3,594 CITATIONS

[SEE PROFILE](#)

Towards a Fuzzy based Framework for Effort Estimation in Agile Software Development

Atef Tayh Raslan

Department of Computer and
Information Sciences, Institute of
Statistical Studies and Research,
Cairo University, Egypt

Nagy Ramadan Darwish

Department of Computer and
Information Sciences, Institute of
Statistical Studies and Research,
Cairo University, Egypt

Hesham Ahmed Hefny

Department of Computer and
Information Sciences, Institute of
Statistical Studies and Research,
Cairo University, Egypt

Abstract—Effort estimation in the domain of software development is a process of forecasting the amount of effort expressed in persons/month required to develop software. Most of the existing effort estimation techniques are suitable for traditional software projects. The nature of agile software projects is different from traditional software projects; therefore using the traditional effort estimation techniques can produce inaccurate estimation. Agile software projects require an innovated effort estimation framework to help in producing accurate estimation. The main focus of this paper is the utilization of fuzzy logic in improving the effort estimation accuracy using the user stories by characterizing inputs parameters using trapezoidal membership functions. In this paper, the researchers proposed a framework based on the fuzzy logic which receives fuzzy input parameters of Story Points (SP), Implementation Level Factor (ILF), FRiction factors (FR), and Dynamic Forces (DF) to be processed in many successive steps to produce in final the effort estimation. The researchers designed the proposed framework using MATLAB to make it ready for later experiments using real data sets.

Keywords: Agile software development, Effort estimation, story points, fuzzy logic

I. INTRODUCTION

Software project management is the process of planning, organizing, staffing, monitoring, controlling and leading a software project. Project life cycle consists of four phases [9, 14]: Project initiation, project planning, project execution, and project closure. Project planning is a crucial phase in software project life cycle because it includes many challenging activities that are necessary; such as software estimation process that includes estimating the size of the software product to be produced, estimating the effort required, developing preliminary project schedules, and finally, estimating overall cost of the project.

Effort estimation is the process of predicting the amount of effort that is expressed by the number of persons per month required to develop or maintain software projects based on incomplete and uncertain requirements. Effort estimates may be used as input to project plans, iteration plans, budgets, and investment analyses.

Software development is a mentally complicated task [25]. Therefore, different software development methodologies and quality assurance methods are used in order to attain high quality, reliable, and bug free software [18]. In recent years, agile software development methods have gained much attention in the field of software engineering [32]. Agile methodologies emphasize on working software as the primary measure of progress [18, 4]. Agile methods deal with unstable and volatile requirements by using a number of techniques, focusing on collaboration between teamwork and customers and support early product delivery [4]. The traditional effort estimation techniques require modifications or improvements to be suitable for agile software methodologies. Therefore, this paper aims to propose an innovated effort estimation framework to help in producing accurate estimation. The proposed framework depends on utilization of fuzzy logic, SP, ILF, FR, and DF.

The rest of this paper is divided into seven sections. Section II introduces an overview on agile software development that includes main characteristics of agile software process and examples of agile methods. Section III presents a brief introduction of fuzzy logic. Section IV introduces a literature review that includes some important papers and researches in effort estimation. Section V introduces effort estimation techniques and focuses on story points. Section VI introduces the proposed framework and explains the main steps required to reach a final effort estimation using this framework. Section VII introduces the conclusion of the paper including the main ideas discussed in the paper. Section VIII introduces the ideas that are expected to be focused in the future.

II. AGILE SOFTWARE DEVELOPMENT

Agile Software Development (ASD) is a group of software development processes that are iterative, incremental, self-organizing and emergent [10]. In addition, it can be defined as a connotation of flexibility, nimbleness, readiness for motion, activity, dexterity in motion, and adjustability [17]. Agile methodologies are a lightweight, efficient, low-risk, flexible, predictable, scientific, and fun way to develop software. In ASD, the time and resources are available which is generally

considered to be fixed [17]. In traditional methods, the time and resources are flexible while the functionality is considered fixed in comparison with the agile one [17]. The agile Manifesto was created in February 2001 that included the twelve principles on agile software development [7]. There are several agile methods based on the idea of agile manifesto. Examples of such methods are Dynamic Systems Development Method (DSDM), eXtreme Programming (XP), Feature Driven Development (FDD) and SCRUM [4, 10, 13, 17, 19, 24, 29].

The characteristics of ASD process include: modularity on development process level, iterative with short cycles, time-bound with iteration cycles, economic in development process, adaptive, incremental, minimize the risks, people oriented, and collaborative and communicative [10,17]. The advantages of agile software development include: Revenue, quality, visibility, risk management, agility, customer satisfaction and help to generate the right product. The revenue refers to the incremental nature of agile development enabling to be realized early as the product continues to develop. The visibility refers to encourage throughout the product development and a very cooperative approach. In addition, in ASD the small incremental releases are visible to the product owner and product team which help them to identify any issues early and make it easier to respond to change.

In traditional methodologies, a team member's workload capacity is determined by the manager who estimates how long certain tasks will take and then assigns work based on that team member's total available time. On other hand, in agile methodology the works are assigned to an entire team, not to an individual member. In addition, agile methodology adopts self-organization of team as a success factor that must be encouraged. Figure 1 shows the effort estimation in scrum methodology which takes place in the pre-game phase. In the sprint planning meeting, the team sits down to estimate its effort for the stories in the backlog. The team shares the concepts and scales that were learned. The product owner needs these estimates to prioritize items in the backlog, and forecast releases based on the team's velocity [33].

III. FUZZY LOGIC

Fuzzy Logic (FL) is a methodology to solve problems which are too complex to be understood quantitatively [5]. It is based on fuzzy set theory and introduced in 1965 by Lotfy Zadeh [1]. The Fuzzy Logic System deals with fuzzy parameters, which address imprecision and uncertainties, by mapping out the path of a given input to an output using the computing framework called the Fuzzy Inference System (FIS). A fuzzy Inference system is a knowledge based or rule based system. FIS consists of four components they are: Fuzzifier, Fuzzy Rule Base, Fuzzy Inference Engine, and Defuzzification. The fuzzifier converts the crisp input into a fuzzy set that is a non-traditional type of sets which allows an element to have a partial degree of membership. The membership refers to the degree of inclusion to specific set .There is many membership

functions but in this research we will use the triangle membership function is represented by Eq. (1) in below [12]:

$$\text{Triangle}(x: a, b, c) = \max \left(\min \left(\frac{x-a}{b-a}, \frac{c-x}{c-b} \right), 0 \right) \quad (1)$$

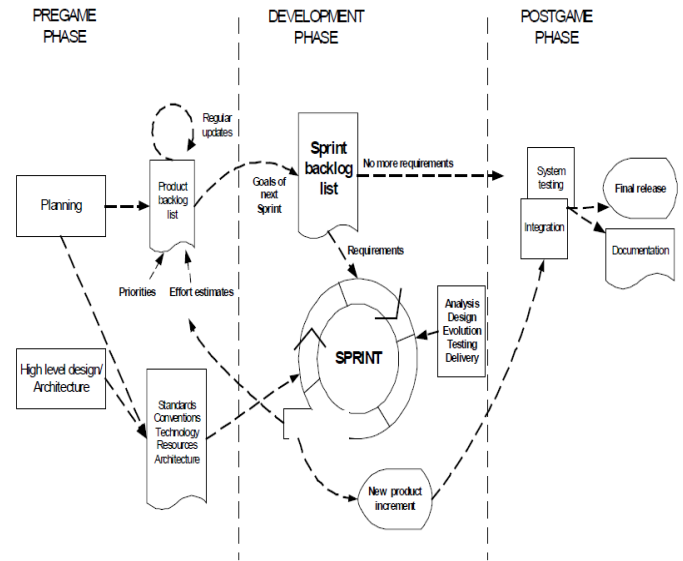
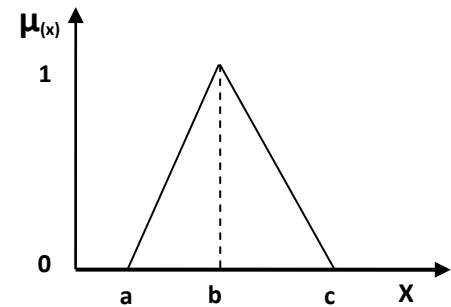


Figure 1: Scrum process [29]

Figure 2 shows the shape of the triangular membership function.



$$\text{Triangle}(x: a, b, c) = \begin{cases} 0 & x < a \\ \frac{x-a}{b-a} & a \leq x \leq b \\ \frac{c-x}{c-b} & b \leq x \leq c \\ 0 & x > c \end{cases}$$

Figure 2: Triangular membership function[12]

A fuzzy inference engine is a collection of IF -THEN rules stored in fuzzy rule base is known as inference engine. Defuzzification is the process which refers to the transform of fuzzy output into crisp output. The most common tools that is used for fuzzy systems is MATLAB which used for defining the input, output, inference rules, and the shape of membership function for the fuzzy system.

Software effort estimation is one of the most challenging activities in project development phases [22, 34]. However, the process of effort estimation is uncertain in nature as it largely depends upon some attributes that are quite unclear during the early stages of development. Fuzzy logic provides the concept of fuzzy sets to handle vague and inaccurate data [1].

IV. LITERATURE REVIEW

There are many papers and researches in effort estimation. The following are some examples of these literatures:

- Masateru T., et al, introduced revisiting software development effort estimation based on early phase development activities. This research aims to examine the relationship of early phase effort and software size with software development effort. The proposed model was constructed using early phase effort and software size. This model was evaluated by using the International Software Benchmarking Standards Group (ISBSG) dataset that collected from software development companies. The result of the experiment showed that when both software size and sum of effort needed for planning and requirement analysis phase were used as explanatory variables, the estimation accuracy was most improved [23].
- Andreas S., et al, introduced a guide to estimates the effort in agile software development. This study provides an investigation about estimation possibilities, especially for XP. It is focused on characteristics of agile methodologies. This study provides guidelines for measurement aspects within XP project. The proposed guide was evaluated using a survey which consists of 17 questions. Each question considers reflect the agile methodologies and effort estimation. The survey shows that, the benefit of agile methods is difficult to assess at the moment. Especially the costs of maintenance projects must be taken into account for it [3].
- Amrita R., et al, introduced a guide to optimizing the cost and effort in scrum projects using function point with COCOMO model. The proposed guide used on the real case study Xsset for estimation. Furthermore, it is show the different tracking tools and representation used to optimize the performance like run chart, JIRA, SVN and SCTM. This research was concluded the agile methodology is very effective in software development. Also, they concluded that by merging the two estimation models can be improving the effort applied and saving the cost [2].
- Shahrukh Z., et al, introduced an effort estimation model for agile software development to accommodate most of the characteristics of agile methodology, especially adaption and iteration, where it is focused on user stories of as base for estimation. The model was evaluated using the empirical data collected from 21 software projects. The experimental results show that model has good estimation accuracy in terms of the Mean Magnitude of Relative Error (MMRE) and probability of a project having a relative error of less than or equal to L PRED (L). MMRE and PRED are computed from the relative error, which is the relative size of the difference between the actual and estimated value of individual effort [33].
- Ziauddin A., et al, introduced a fuzzy logic based software cost estimation Model. This study aims to utilize a fuzzy logic model to improve the accuracy of software effort estimation. The main idea in this study is fuzzifying input parameters of COCOMO II model and the result is defuzzified to get the resultant Effort. Triangular fuzzy numbers are used to represent the linguistic terms in COCOMO II model. The results of this model are compared with COCOMO II and Alaa Sheta Model. The proposed model yields better results in terms of MMRE, PRED (n) and Variance Account for (VAF). VAF is used in the context of statistical models whose main purpose is the prediction of future outcomes on the basis of other related information [34].
- Vishal S., et al, introduced optimized fuzzy logic based framework for effort estimation in software development. The performance of the proposed framework is demonstrated in terms of empirical validation carried on live project data of the COCOMO public database. The performance of the framework is evaluated using live project data of the COCOMO public database. It is shows that the proposed framework can be deployed on COCOMO II environment with experts providing required information for developing fuzzy sets and an appropriate rule base [30].
- Ratnesh L. and Abhay K., introduced an estimation model for agile web based that aim to address the numerous open issues in web development particularly in context to agile software development by analyzing data from completed agile Web development projects. This study identifies the difference between traditional and web projects. Furthermore, it is offers a tool for effort estimation based on agile manifestoes and web characteristics [26].
- Abeer H. introduced a model based on a fuzzy logic for enhancing the sensitivity of COCOMO cost model. This

study enhances the accuracy and sensitivity of COCOMO 81 intermediate by fuzzifying the cost drivers. This model was implemented through the MATLAB. The dataset was collected from six NASA centers and covers a wide range of software domains, development process, languages and complexity, as well as fundamental differences in culture and business practices between each centre. The results showed that the sensitivity of the proposed fuzzy model is superior to COCOMO81 intermediate [1].

- Jitender Choudharia and Ugrasen Suman, introduced software Maintenance Effort Estimation Model (SMEEM) for software maintenance estimation. The proposed model uses SP to calculate the volume of maintenance and value adjustment factors that are affecting story points for effort estimation. The proposed model is illustrated with various types of maintenance projects such as web based application, MIS and critical project. It is designed to help the project manager in charge of software maintenance to calculate the estimated software maintenance effort in terms of Adjusted Story Point (ASP), size, cost and duration. It generates the more realistic and precise estimation results. This model is applicable only for agile and extreme programming based maintenance environment [16].

V. EFFORT ESTIMATION TECHNIQUES

Software effort estimation techniques can be classified into algorithmic and non-algorithmic models [26]. The most popular models that were classified as non-algorithmic technique are:

- *Expert judgment.*
- *Thumbs rule.*
- *Delphi technique.*
- *Wide band delphi technique.*
- *Parkinson's law.*
- *Pricing to win.*
- *Probe software process.*
- *Team software process technique planning.*

The recent researches focused on an algorithmic model such as [6, 20, 26]:

- *Line Of Code (LOC).*
- *Functions point matrices.*
- *COCOMO and COCOMO-II.*
- *Object points.*
- *Software life cycle management.*
- *Use case estimation.*
- *Story points [8,11,33].*

Algorithmic models are based on the statistical analysis of historical data. These models require accurate input of specific attribute such as line of code LOC, function points FP, number of user screens, complexity, and velocity [8, 33].

In this paper, the researchers focused on the story point for estimating the efforts in agile software development. The story point is the most common estimation technique for agile software development [8, 11, 33].

Story points refer to an estimate of the relative scale of the work in terms of actual development effort. Story points are usually expressed either in numbers that follow the Fibonacci series, t-shirt sizes (XS, S, M, L, XL, XXL) or even dog breeds (Chihuahua to Great Dane) [33]. Fibonacci series are a sequence of numbers where each number is the sum of the previous two. Fibonacci numbers are 1, 2, 3, 5, 8, 13, 21, etc. Story point estimation is done using relative sizing by comparing one story with a sample set of previously sized stories. Relative sizing across stories tends to be much more accurate over a larger sample, than trying to estimate each individual story for the effort involved. Planning poker is one of the most useful tools in agile software development [31].

In ASD, each member of the team is given a group of index cards. Each index card represents a size in user story sequence. Each member of the team then chooses a card representing their estimate of the effort involved in completing a particular user story, and places that card face down in front of them.

The agile effort estimation includes story size, complexity, implementation level factor, and velocity [8]. Story size refers to an estimate of the relative scale of the work in terms of actual development effort. The values assigned are usually taken from the Fibonacci sequence. The assigned values can be changed by the team itself or even the criteria can be redefined. The Complexity (Com) scale assigned like a story size by using Fibonacci sequence. The user stories grouped according to the characteristics of agile methods [21]. Also, these groups can be adjusted by the team itself.

The velocity refers to how much product backlog effort a team can handle in one unit of time. Actual velocity of any development project varies with the size and experience of a team [34]. Computing velocity helps the team to improve their estimates over the life span of a project. In addition, the velocity refers to the team's capacity which enables release planning and over time calculations.

Optimization process refers to studying the project constraints which should be completed before the calibration to improve the stability of the velocity calculation. This process includes two factors they are FR and DF. The friction includes a range of factors that may affect the team velocity they are team composition, process, environmental factors, and team dynamic [15]. The composition of the team concerns the skills and attitudes of team members. The process factor refers to the percentage of change in the agile methods, release, building and testing. The environmental factors refer to the noise, poor ventilation, poor lighting, uncomfortable seating and desks, inadequate hardware or software. Team dynamics are patterns of interaction among team members that determine the performance of the team.

Table 1 shows the FR with a range of values. Each factor has been adjusted according to their risk level (stable, volatile, highly volatile, and very highly volatile).

TABLE 1: FRICTION FACTORS [33]

Friction factors	Stable	volatile	Highly volatile	Very highly volatile
Team composition	1	.98	.95	.91
Process	1	.98	.94	.89
Environmental factors	1	.99	.98	.96
Team dynamic	1	.98	.91	.85

DF refers to the factors that could lead to loss of velocity [33]. These factors are unpredictable and unexpected. DF including nine factors: team changes, new tools, vendor defects, responsibilities out of the project, personal issues, stakeholders, unclear requirements, changing requirements, and reallocation. Those factors are ranked from (Normal, high, very high, and extra high). Table 2 shows the assigned values for those factors.

TABLE 2: VARIABLE FACTORS [33]

Variable factors	Normal	High	Very High	Extra High
Expected to team change	1	.98	.95	.91
Introduction to a new tools	1	.99	.97	.96
Vendor's Defect	1	.98	.94	.90
Team member's responsibilities outside the project	1	.99	.98	.98
Personal Issues	1	.99	.99	.98
Expected Delay in Stakeholder response	1	.99	.98	.96
Expected Ambiguity in Details	1	.98	.97	.95
Expected Changes in environment	1	.99	.98	.97
Expected Relocation	1	.99	.99	.98

VI. PROPOSED FRAMEWORK

The process of effort estimation is uncertain in nature as it largely depends upon some attributes that are quite unclear during the early stages of development [26]. Fuzzy logic provides the concept of fuzzy sets to handle vague and inaccurate data. Its address imprecision and uncertainties, by mapping out the path of a given input to an output using the computing framework called the FIS. It consists of four main components; they are: Fuzzifier, fuzzy rule base, fuzzy inference engine, and defuzzification. The fuzzifier converts the crisp input into a fuzzy set by using a membership function. Fuzzy rule base which represented by if-then rules. Fuzzy

inference engine is a collection of if-then rules. Defuzzification is the process that refers to the translation of fuzzy output into crisp output. The MATLAB Fuzzy Inference System was used in the fuzzy calculations.

Figure 3 shows that the flowchart of the proposed framework starts with inputs *SP*, *ILF*, *FR*, and *DF*. Then, these inputs are fuzzified to obtain fuzzy sets using the triangular member function. Rule base includes a group of IF-Then rules that define the complexity and velocity fuzzy variables. The FIS evaluates all rules of the rule base. The resulted fuzzy set is deuzzified to a crisp value and then the complexity and velocity are calculated. Finally, the effort is estimated using the resulted complexity and velocity.

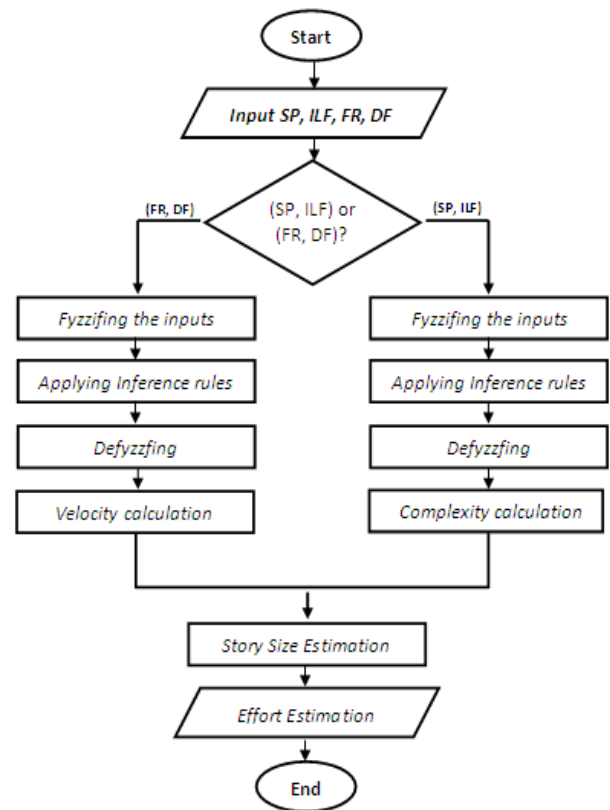


Figure 3: Proposed framework Flowchart

Figure 4 shows the proposed framework for effort estimation in agile project based on fuzzy logic which consists of three phases: The first phase refers to input that includes three inputs *SP*, *ILF*, and *Com*. These input variables are changed to fuzzy variables based on fuzzification process. In the story point the terms Very Small Story (VSS), Small Story (SS), Medium Story (MS), Large Story (LS), and Very Large Story (VLS) were defined for that input.

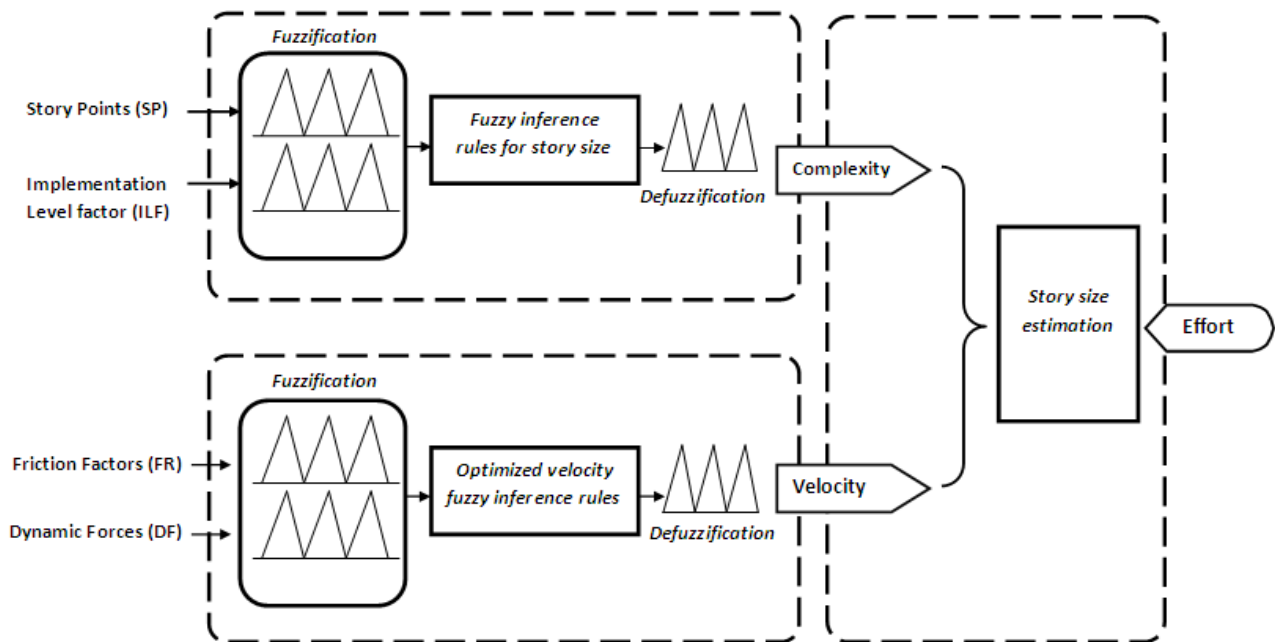


Figure 4: Proposed Framework for Effort Estimation ASD

Table 3 shows the weight of SP which can have many iterations depending upon the system requirements. The weights assigned for each user stories are predicted by using Fibonacci sequence.

TABLE 3: SP WEIGHTS

Term	Wight
Very Small Story (VSS)	1
Small Story (SS)	2
Medium Story (MS)	3
Large Story (LS)	5
Very Large Story (VLS)	8

Figure 5 shows the linguistic variables for a story point which has a range of weight (VSS, SS, MS, LS, and VLS).

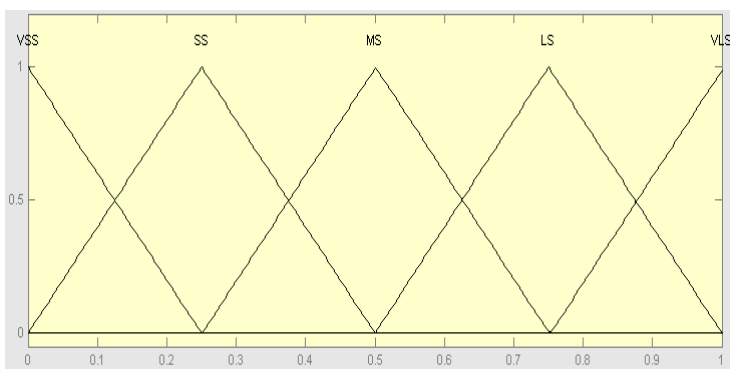


Figure 5: Linguistic variables for SP

In ILF includes the terms Off the Shelf (OS), Full Experience Components (FEC), Partial Experience Components (PEC), and New Components (NC). Table 4 shows the scaling factor for implementation level which represents the level of understanding each user story.

TABLE 4: ILF SCALING

Term	Scale
Off the Shelf (OS)	1
Full Experience Components (FEC)	2
Partial Experience Components (PEC)	3
New Components (NC)	4

Figure 6 shows the linguistic variables for implementation level which has a range of weight (OS, FEC, PEC, and NC).

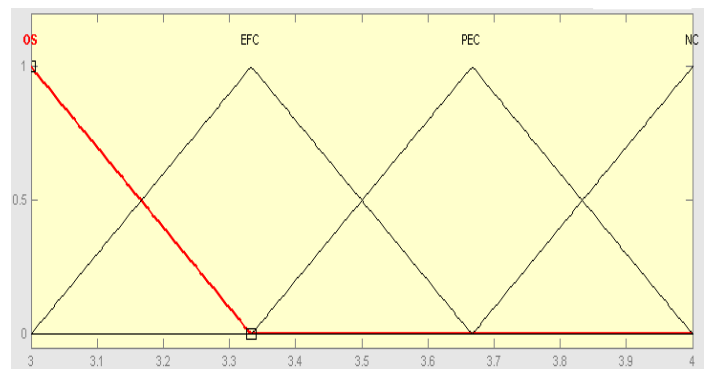


Figure 6: Linguistic variables for ILF

Table 5 shows the Com in the terms Very Low (VL), Low (L), Medium (M), High (H), and Very high (VH). The complexity has been determined based on the user stories in ILF. For example, if the user story is very low and ILF is OS, then Com is very low.

TABLE 5: COM LEVELS

Term	Level
Very Small Story (VSS)	1
Small Story (SS)	2
Medium Story (MS)	3
Large Story (LS)	5
Very Large Story (VLS)	8

Figure 7 shows the linguistic variables for a complexity which has a range of weight (VL, L, M, H, and VH).

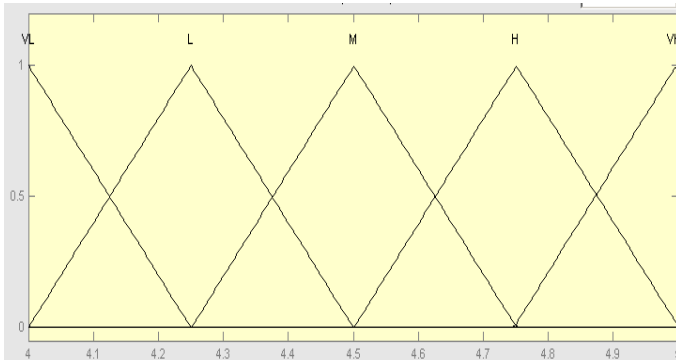


Figure 7: Linguistic variables for Com

The effort for the complete project will be the sum of membership degree of SP, ILF, and Com for all individual user stories. The Eq. (2) show the fuzzy story size (FSS) by using the triangle membership function μ .

$$FSS = SP \cdot \mu_{SP} \times \sum_i^4 ILF \cdot \mu_i^{ILF} \times \sum_i^5 Com \cdot \mu_i^{Com} \quad (2)$$

FR includes the terms Stable(S), Volatile (V), Highly Volatile (HV), and Very Highly Volatile (VHV) were defined for the four variables team composition, process, environmental factors, and team dynamic. Figure 8 shows the linguistic variables for a team composition which has a range of weight (S, V, HV, and VHV).

DF includes the terms Normal (N), High (H), Very High (VH), and Extra High (EH) were defined for the nine variables they are: Expected to team change, introduction to new tools, vendor's defect, team member's responsibility outside the project, personal issues, expected delay in stakeholder response, expected ambiguity in details, expected changes in environment, and expected relocation. Figure 9 shows the

linguistic variables for expected to team change which has a range of weight (N, H, VH, and EH).

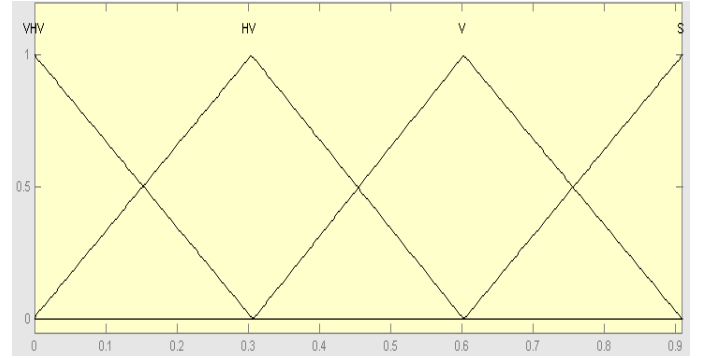


Figure 8: Linguistic variables for FR

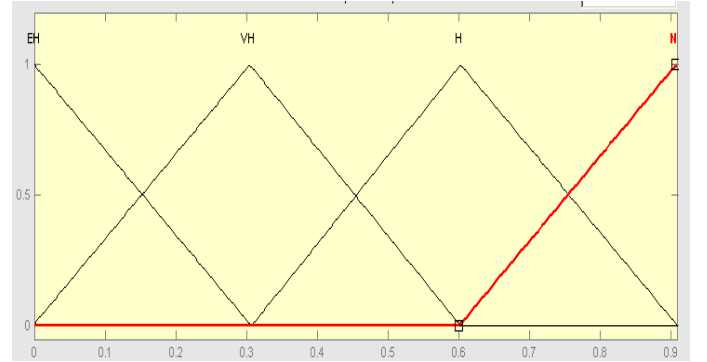


Figure 9: Linguistic variables for DF

Fuzzy FR (FFR) is calculated as product of all four components of FR and the membership degree. Eq.3 Shows the FFR calculation by using the triangle membership function μ :

$$FFR = \prod_{i=1}^4 FR \cdot \mu_i^{FF} \quad (3)$$

The Deceleration (D) represents the rate of negative change of velocity. The D and adjusted Velocity (V) are calculated as [33]:

$$D = FR \times DF \quad (4)$$

$$V = (VI)^D \quad (5)$$

The compilation time (T) is calculates by the following formula [33]:

$$T = \frac{E}{V} \quad (6)$$

The second phase is inference engine which run on a set of fuzzy rules. The Fuzzy Model rules contain the linguistic variables related to the agile project. This is sample of the rules that represent the proposed framework below:

IF SP is VSS and ILF is OS THEN Com is VL
IF SP is VLS and ILF is NC THEN Com is VLS
IF SP is MS and ILF is OS THEN Com is SS
IF SP is LS and ILF is PEC THEN Com VLS

The third phase a defuzzification process which is calculates and converts the fuzzy output into crisp data form. The most common method is a Centeroid Method or it is called Center Of Area (COA) [34].

$$COA = \frac{\sum_{x=a}^B \mu(X).X}{\sum_{x=a}^B \mu(X)}$$

VII. CONCLUSION

This paper aimed to propose a fuzzy based framework for effort estimation in agile software development. Therefore, the researchers studied many researches in the domain of effort estimation of agile software development either based on fuzzy logic or not. In addition, the researchers studied in detail the features of agile software development, story points, and fuzzy logic. Then, the researchers proposed a framework that is based on the fuzzy logic which receives fuzzy input parameters of story points, implementation level factor, friction factors, and dynamic forces. The proposed framework starts with inputs *SP*, *ILF*, *FR*, and *DF*. Then, these inputs are fuzzified to obtain fuzzy sets using the triangular member function. Rule base includes a group of IF-Then rules that define the complexity and velocity fuzzy variables. The fuzzy inference system evaluates all rules of the rule base. The resulted fuzzy set is deuzzified to a crisp value and then the complexity and velocity is calculated. Finally, the effort is estimated using the resulted complexity and velocity.

The researchers thought that the utilization of fuzzy logic and story point in the proposed framework may help in producing an accurate estimation of effort. The researchers designed the proposed framework using MATLAB to make it ready for later experiments using real data sets.

VIII. FUTURE WORK

The ideas that are expected to be focused in the future include:

- Applying the proposed framework in this paper using real data set from software projects and comparing the results with other results produced from different techniques.
- Enhancing the framework using the results of the application.

- Utilization of COCOMOII to enhance the proposed framework

REFERENCES

- [1] Abeer Hamdy, "Fuzzy Logic for Enhancing the Sensitivity of COCOMO Cost Model", Journal of Emerging Trends in Computing and Information Sciences, Vol.3, No.9, 2012.
- [2] Amrita Raj Mukker, Anil Kumar Mishra, and Latika Singh, "Enhancing Quality in Scrum Software Projects", International Journal of Science and Research (IJSR), Vol. 3, Issue 4, 2014.
- [3] Andreas Schmietendorf, Martin Kunz, and Reiner Dumke, "Effort estimation for Agile Software Development Projects", 5th Software Measurement European Forum, 2008.
- [4] Ann L. Fruhling and Alvin E. Tarrell, "Best Practices for Implementing Agile Methods", IBM, 2008.
- [5] Anupama Kaushik, A.K. Soni, and Rachna Soni, "A Type-2 Fuzzy Logic Based Framework for Function Points",
- [6] Ashita Malik, Varun Pandey, Anupama Kaushik, "An Analysis of Fuzzy Approaches for COCOMO II", International Journal of Intelligent Systems and Applications, Vol.5, 2013.
- [7] Beck et al., "http://agilemanifesto.org/iso/en/manifesto.html," last visited in December 2014.
- [8] C. Sathish Kumar, A. Anitha Kumari, and R. Srinivasa Perumal, "An Optimized Agile Estimation Plan Using Harmony Search Algorithm," International Journal of Engineering and Technology (IJET), Vol 6 No 5, 2014.
- [9] Daniele Di Filippo, Russell D. Archibald, and Ivano Di Filippo, "Six-Phase Comprehensive Project Life Cycle Model", Vol. I, Issue V, December 2012.
- [10] David Cohen, Mikael Lindvall, Patricia Costa, "An Introduction to Agile Methods," Advances in computers, Elsevier, V. 62, pp. 20-22, 2004.
- [11] Evita Coelho, Anirban Basu, "Effort Estimation in Agile Software Development using Story Points," International Journal of Applied Information Systems (IAIS), Volume 3- No.7, August 2012.
- [12] Ganesh M., "Introduction to Fuzzy Sets and Fuzzy Logic", Prentice-Hall, 978-8120328617, 2006.
- [13] Ioannis G. Stamelos, and Panagiotis Sfetos, "Agile software development quality assurance", Idea Group, ISBN 978-1-59904-216-9, 2007.
- [14] Jason Westland, "Project management guidebook", Method123, 2003.
- [15] Jeff Dyer, Jeffrey H. Dyer, William G., "Team Building: Proven Strategies for Improving Team Performance", Jossey-Bass, ISBN: 978-1118416143, 2013.
- [16] Jitender Choudharia and Ugrasen Suman, "Story Points Based Effort Estimation Model for Software Maintenance", Elsevier, Procedia Technology, Volume 4, 2012.
- [17] John Hunt, "Agile software construction", Springer, ISBN-10: 1-85233-944-6, 2006.
- [18] K.Usha, N.Poonguzhali, and E.Kavitha, "A Quantitative Approach for Evaluating the Effectiveness of Refactoring in Software Development Process", International Conference on Methods and Models in Computer Science, Delhi, India, Dec. 2009.
- [19] Kevin Vlaanderen, Sjaak Brinkkemper, Slinger Jansen, Erik Jaspers, "Applying SCRUM Principles to Software Product Management", Information and Software Technology, V. 53, Issue 1, pp. 58-70, 2011.
- [20] Komal Garg, Paramjeet Kaur, Shalini Kapoor, and Shilpa Narula, "Enhancement in COCOMO Model Using Function Point Analysis to Increase Effort Estimation", IJCSMC, Vol. 3, Issue. 6, 2014.
- [21] Luigi Buglione, and Alain Abran, "Improving Estimations in Agile Projects: Issues and Avenues", Software Measurement European Forum (SMEF), 2007.
- [22] M.C. Ohlsson, C. Wohlin, and B. Regnell, "A project effort estimation study", Elsevier Science, information and software technology, Vol.40, 831- 839, 1998.

- [23] Masateru Tsunoda, Koji Toda, and Kyohei Fushida, Yasutaka Kamei, Meiyappan Nagappan, and Naoyasu Ubayashi, "Revisiting Software Development Effort Estimation Based on Early Phase Development Activities", 10th conference on mining software repositories (MSR), IEEE, 2013.
- [24] Mike Cohn, "Software Development Using Scrum," Addison-Wesley, ISBN: 978-0321579362, 2009.
- [25] Nagy Ramadan Darwish, "Improving the Quality of Applying eXtreme Programming (XP) Approach", International Journal of Computer Science and Information Security (IJCSIS) – ISSN 1947-5500, Vol. 9 No. 11, November 2011.
- [26] Ratnesh Litoriya and Abhay Kothari "An Efficient Approach for Agile Web Based Project Estimation: AgileMOW", Journal of Software Engineering and Applications, VOL.6, 2013.
- [27] Sandeep Kad, Vinay Chopra, "Fuzzy Logic based framework for Software Development Effort Estimation", An International Journal of Engineering Sciences (IJES), Vol. 1, 2011.
- [28] Siva Dorairaj, James Noble, and Petra Malik, "Understanding Team Dynamics in Distributed Agile Software Development", Springer, ISBN: 978-3-642-30350-0, 2012.
- [29] Warsta J., Ronkainen J., Salo O., Abrahamsson P., "agile software development", VTT, ISBN 951-38-6009-4, 2002.
- [30] Vishal Sharma and Harsh Kumar Verma, "Optimized Fuzzy Logic Based Framework for Effort Estimation in Software Development", International Journal of Computer Science Issues (IJCSI), Vol. 7, Issue 2, No 2, 2010.
- [31] Vitaliy Zurian, "http://www.agilemarketing.net/winning-planning-poker/", last visited on June 2015.
- [32] Yang Yong and Bosheng Zhou, "Evaluating Extreme Programming Effect through System Dynamics Modeling", International Conference on Computational Intelligence and Software Engineering (CiSE), Wuhan, China, Dec. 2009.
- [33] Ziauddin Zia, Shahid Kamal Tipu, and Shahrukh Zia "An Effort Estimation Model for Agile Software Development", Advances in Computer Science and its Applications (ACSA), Vol.2, 2012.
- [34] Ziauddin, Shahid Kamal, Shafiullah Khan and Jamal Abdul Nasir, "A Fuzzy Logic Based Software Cost Estimation Model", International Journal of Software Engineering and Its Applications (IJSEIA), Vol. 7, Issue 2, 2013.